



**Tecnológico
de Monterrey**

Campus Santa Fe

I. Introducción y Problemática

Los sistemas de bicicletas compartidas, como **Ecobici en la Ciudad de México**, se han consolidado como un pilar fundamental de la movilidad urbana. Sin embargo, estos sistemas se enfrentan de manera crónica al **fenómeno de la asimetría espacio-temporal de la demanda**, comúnmente conocido como el reto del rebalanceo de unidades.

Esta problemática es impulsada por patrones de movilidad pendular altamente predecibles pero difíciles de mitigar de forma pasiva. Durante las mañanas, los flujos de usuarios se concentran masivamente desde las zonas residenciales periféricas hacia los distritos financieros y de oficinas. Por las tardes, el fenómeno se invierte de manera abrupta.

Actualmente, los operadores logísticos de Ecobici ejecutan un rebalanceo basado en horarios fijos preestablecidos que no reaccionan a las necesidades de bicis de los usuarios del sistema

Para resolver esto, es indispensable una transición hacia un **rebalanceo predictivo**. El núcleo de este proyecto radica en anticipar con un **horizonte temporal objetivo de 50 minutos** el estado exacto de ocupación de las estaciones. Este margen de 50 minutos proporciona la ventana de tiempo óptima y realista para que la infraestructura logística urbana procese las órdenes de trabajo, despache los vehículos de rebalanceo y mitigue proactivamente los eventos de saturación o desabasto antes de que afecten la experiencia del usuario.

II. Arquitectura de la Solución

Una instancia de Backend (FastAPI) intercepta, valida y persiste el JSON en una Base de Datos PostgreSQL alojada en la nube para su historial y auditoría. El backend expone además una API REST que sirve los datos al frontend, gestiona la autenticación de usuarios mediante JWT y ejecuta de forma autónoma una nueva predicción cada cierto intervalo de tiempo mediante un scheduler integrado.

1. On premise

El flujo comienza con el backend de la nube llama cada 10 minutos el modelo predicción preentrenado que consume la API de Ecobici. Tras la recolección de los datos en un buffer local, el nodo local consolida las variables que usara el modelo y lo ejecuta.

- **Regresión:** Un modelo **XGBRegressor** estima el volumen numérico de bicicletas a 50 minutos a futuro.
- **Clasificación:** Un **XGBClassifier** multiclase toma dicha predicción y categoriza la estación en uno de cuatro estados operativos (Déficit, Aceptable Bajo, Aceptable Alto o Saturación).
- **Optimización Logística:** Ante estados críticos (Déficit o Saturación), el **Algoritmo de Ruteo** calcula las matrices de Distancia Manhattan para buscar una estación cercana que pueda saciar esa necesidad.

Al concluir el servidor local serializa un payload JSON y lo transmite hacia la nube.

2. Cloud

Una instancia de Backend dedicada intercepta, valida y almacena el JSON en una Base de Datos Cloud para su persistencia histórica y auditoría.

Las peticiones de los usuarios web ingresan a través de un Balanceador de Carga que distribuyendo el tráfico hacia dos instancias Frontend. Dando finalmente acceso a la página web.

III. Modelo de Machine Learning

1. Arquitectura del Modelo Predictivo Híbrido

El sistema utiliza un enfoque secuencial e híbrido que combina regresión avanzada y clasificación multiclase mediante algoritmos de Gradient Boosting (**XGBoost**) para anticipar las necesidades operativas de la red de estaciones con un horizonte temporal objetivo de 50 minutos.

1.1 Ingeniería de Features con Inercia Orgánica

Se genera memoria de manera dinámica interactuando con un buffer local en disco (`buffer_tiempo_real.csv`). Cada 10 minutos, el script procesa un vector alineado de 27 variables divididas en los siguientes bloques:

- **Variables Temporales Cíclicas:** Transformación de la hora del sistema mediante funciones trigonométricas (sin y cos) para representar la continuidad del tiempo en el cambio de día, junto con componentes de calendario y segmentación laboral (días laborables, fines de semana).
- **Estructuras de Retraso Orgánicas (Lags):** Captura de los estados reales de ocupación (`pct_full`) de los últimos 10, 20, 30 y 60 minutos extraídos del buffer flotante.
- **Dinámica de Flujos y Segundas Derivadas:** Cálculo numérico instantáneo de la velocidad *de cambio* (primer gradiente) y aceleración *de cambio* (segundo gradiente) en las estaciones para identificar tendencias de vaciado o llenado descontrolado.
- **Ventanas Móviles Estadísticas:** Métricas de tendencia central y dispersión (`rolling_mean` y `rolling_std`) basadas en el pasado inmediato real (últimas ventanas de 30 y 60 minutos).
- **Conocimiento Contextualizado Endógeno:** Incorporación de tablas de búsqueda agregadas (*Target Encoding*) provenientes del entrenamiento histórico para ligar cada `station_id` a su comportamiento típico según hora y tipo de día.
- **Restricciones Físicas e Interacciones:** Capacidad estructural de la estación, espacios disponibles en tiempo real e interacciones multiplicativas entre infraestructura y tiempo.
- **Segmentación Espacial:** Agrupación geográfica por clústeres asignados estáticamente a través de una coordenada base.

1.2 Predicción de Volumen

- **Objetivo:** Estimar de manera continua el número exacto de bicicletas disponibles que tendrá cada estación en los siguientes 50 minutos.

- **Algoritmo:** XGBRegressor optimizado con base al MAE para mitigar el impacto de valores atípicos en la demanda urbana. El resultado pasa por un acotamiento físico estricto (`np.clip`) restringido entre 0 y la capacidad total del rack.

1.3 Clasificación de Nivel

- **Objetivo:** Categorizar el estado futuro de la estación para determinar la urgencia y el sentido de la intervención logística (sumar o retirar unidades).
- **Algoritmo:** XGBClassifier secuencial multiclase. Toma como variable de entrada adicional la predicción numérica generada por el regresor.

1.4 Rangos de Ocupación Estrictos

- **Clase 0 (Déficit Crítico):** Ocupación $\leq 30\%$. La estación corre riesgo inminente de quedarse vacía. Requiere inyección de bicicletas desde puntos de oferta o almacén.
- **Clase 1 (Aceptable Bajo):** Ocupación $> 30\%$ y $\leq 55\%$. Estado estable. No requiere intervención.
- **Clase 2 (Aceptable Alto):** Ocupación $> 55\%$ y $\leq 85\%$. Estado estable. No requiere intervención.
- **Clase 3 (Saturación Crítica):** Ocupación $> 85\%$. La estación corre riesgo de bloquearse imposibilitando las devoluciones de los usuarios. Requiere retiro de bicicletas.

2. Lógica del Algoritmo de Ruteo Geográfico

Cuando el clasificador determina que una estación entrará en un estado crítico (Clase 0 o Clase 3), el sistema calcula el número neto de unidades a mover. El inventario objetivo se fija exactamente en el **55% de la capacidad total de la estación**, correspondiente al límite superior de control de la Clase 1:

2.1 Circuito Local

El algoritmo agrupa las estaciones críticas por su zona logística. Para cada estación en Déficit (Clase 0) busca estaciones con Saturación (Clase 3) dentro de su misma región. Las órdenes de traslado se emparejan priorizando la proximidad física mediante el cálculo de la **Distancia**

Manhattan urbano. Se establece un límite geográfico estricto de **1 km** para validar la viabilidad del circuito vecinal.

2.2 Captura de Oferta y Demanda

- Si una estación en déficit crítico (Clase 0) no dispone de estaciones vecinas con excedentes en su zona o dentro del radio límite de 1 km, su necesidad se deriva automáticamente como una orden de provisión hacia el **Almacén Central de Ecobici**.
- Si una estación saturada (Clase 3) mantiene excedentes de inventario que no pudieron ser absorbidos por ninguna estación vecina en escasez bajo los mismos criterios geográficos, el sobrante se despacha como una orden de resguardo hacia el **Almacén Central de Ecobici**.

3. Especificaciones del Output (Payload JSON)

El sistema genera un archivo JSON estandarizado.

3.1 Esquema del JSON

```
{  
  "status": "string (Valor fijo: 'success' | 'error')",  
  "timestamp_evaluacion": "string (Formato ISO: 'YYYY-MM-DD HH:MM:SS.ffffff')",  
  "metricas_globales": {  
    "model_performance": {  
      "accuracy_semaforo_pct": "float (Rango: 0.0 a 100.0)",  
      "mae_volumen_bicicletas": "float (Valor >= 0.0)"  
    },  
    "green_logistics": {  
      "movimientos_mitigados_unidades": "integer (Valor >= 0)",  
      "eficiencia_rebalanceo_local_pct": "float (Rango: 0.0 a 100.0)",  
      "distancia_total_optimizada_local_km": "float (Valor >= 0.0)"  
    },  
    "flota_resumen": {
```

```
"Camioneta Ligera (Cap 16 - Sin Remolque)": "integer (Opcional, Conteo >= 0)",
"Camioneta + Remolque (Cap 32)": "integer (Opcional, Conteo >= 0)",
"Camión Grande (Cap 50)": "integer (Opcional, Conteo >= 0)"
}
},
"hoja_de_ruta": [
{
"zonaLogistica": "integer (Identificador discreto)",
"estacionOrigen": "integer (ID de estación) OR string (Valor fijo: 'Almacén Central')",
"estacionDestino": "integer (ID de estación) OR string (Valor fijo: 'Almacén Central')",
"bicicletasAMover": "integer (Valor estricto > 0)",
"distanciaKm": "float (Valor > 0.0) OR null (Cuando origen/destino es 'Almacén
Central')",
"vehiculoAsignado": "string (Valores permitidos: 'Camioneta Ligera (Cap 16 - Sin
Remolque)' | 'Camioneta + Remolque (Cap 32)' | 'Camión Grande (Cap 50)')"
}
]
}
```

IV. Backend

1. Stack Tecnológico

El backend está construido en Python con FastAPI como framework web y Uvicorn como servidor ASGI. La persistencia se maneja con PostgreSQL a través de SQL Model (combinación de SQLAlchemy y Pydantic), lo que permite definir los modelos de base de datos y los esquemas de validación en una misma clase. La autenticación se implementa con JWT firmados mediante HS256 (python-jose) y contraseñas hashadas con bcrypt. Las tareas programadas se gestionan con APScheduler corriendo en segundo plano dentro del mismo proceso.

2. Modelos de Base de Datos

El esquema relacional se compone de cuatro tablas:

Tabla	Descripción
usuario	Cuentas de usuario con nombre, email, hash de contraseña, rol y estado activo/inactivo.
prediccion	Resultado de cada ejecución del modelo: timestamp, métricas globales de logística verde y resumen de flota (serializado en JSON).
ruta	Cada orden de movimiento generada por el algoritmo de ruteo: estación origen, estación destino, bicicletas a mover, distancia, vehículo asignado y estado de completado.
audit_log	Registro inmutable de acciones críticas: logins, ejecuciones de predicción y finalización de rutas, con IP de origen y timestamp.

3. Sistema de Autenticación y Control de Acceso

El sistema implementa autenticación basada en tokens JWT con un esquema de control de acceso por roles (RBAC). Cada token contiene el ID del usuario y su rol, evitando una consulta adicional a la base de datos en cada request.

Se definen tres roles con distintos niveles de acceso:

Rol	Capacidades
admin	Acceso total: gestión de usuarios, auditoría, ejecución de predicciones y cierre de rutas.
operador	Puede ejecutar predicciones manualmente y marcar rutas como completadas.
viewer	Solo lectura: consulta predicciones históricas y estado de estaciones.

4. API REST Endpoints

El backend expone cinco grupos de endpoints. La documentación interactiva (Swagger UI) está disponible en /docs al correr el servidor.

Autenticación /auth

Método	Ruta	Descripción	Acceso
POST	/auth/register	Registrar nuevo usuario (rol viewer por defecto)	Público
POST	/auth/login	Iniciar sesión, retorna JWT	Público
GET	/auth/me	Obtener perfil del usuario autenticado	Autenticado

Predicciones /predicciones

Método	Ruta	Descripción	Acceso
POST	/predicciones/ejecutar	Disparar una predicción manualmente	admin, operador
GET	/predicciones/latest	Predicción más reciente con todas sus rutas	Autenticado
GET	/predicciones	Historial de las últimas 20 predicciones	Autenticado

Rutas de redistribución /rutas

Método	Ruta	Descripción	Acceso
PATCH	/rutas/{id}/completar	Marcar una ruta como ejecutada en campo	admin, operador

Administración /admin

Método	Ruta	Descripción	Acceso
GET	/admin/usuarios	Listar todos los usuarios del sistema	admin
POST	/admin/usuarios	Crear usuario con rol específico	admin
PATCH	/admin/usuarios/{id}	Modificar nombre, rol o estado de un usuario	admin
GET	/admin/audit-log	Consultar el registro de auditoría completo	admin

Estaciones /estaciones

Método	Ruta	Descripción	Acceso
GET	/estaciones	Estado en tiempo real de todas las estaciones Ecobici	Autenticado

5. Integración con el Modelo de Machine Learning

El backend actúa como orquestador de las predicciones. El servicio del modelo corre como un proceso independiente y expone un único endpoint POST /predecir. El backend lo invoca de dos formas:

- Manual: Cuando un usuario con rol admin u operador llama a POST /predicciones/ejecutar.
- Automática: Mediante un scheduler (APScheduler) que se inicializa al arrancar el servidor y dispara la predicción en un intervalo configurable (por defecto cada 5 minutos).

Al recibir la respuesta del modelo, el backend extrae las métricas globales del sub-objeto `metricas_globales.green_logistics`, persiste el resumen de flota como JSON serializado en la tabla `prediccion`, y crea un registro en la tabla `ruta` por cada entrada del arreglo `hoja_de_ruta`. El timeout de la llamada HTTP al modelo es de 60 segundos para absorber la latencia del procesamiento en tiempo real.

6. Datos en Tiempo Real de Estaciones

El endpoint /estaciones no consulta la base de datos local. En cada llamada realiza dos requests síncronos a la API pública GBFS de Ecobici (especificación General Bikeshare Feed Specification), combinando la información estática de estaciones (nombre, coordenadas, capacidad) con el estado dinámico (bicicletas disponibles, andenes libres, estado operativo). El porcentaje de ocupación (`pct_full`) se calcula en el backend como `bicicletas_disponibles /`

capacidad, con un piso de 1 en la capacidad para evitar división por cero en estaciones sin aforo publicado.

Repositorios GitHub

- Machine Learning Model: <https://github.com/tomasmolinatec/modelo-ml/>
- Backend: <https://github.com/Diegoav87/ecobici-backend>
- Frontend: <https://github.com/Diegoav87/ecobici-frontend>

V. Frontend

1. Stack Tecnológico

El frontend es una Single Page Application (SPA) desarrollada en React 19 con TypeScript. Vite actúa como herramienta de build y servidor de desarrollo con Hot Module Replacement. El estilizado se realiza íntegramente con Tailwind CSS 4 sin depender de ninguna librería de componentes externa. La comunicación con el backend se realiza mediante Axios.

2. Estructura del Proyecto

El código fuente se organiza bajo src/ en las siguientes carpetas:

Carpeta / Archivo	Contenido
main.tsx	Punto de entrada: monta la app en el div #root.
App.tsx	Configura el router, el AuthProvider y la Navbar global.
components/	Componentes reutilizables: Navbar y ProtectedRoute.
context/	AuthContext: estado global de autenticación (usuario, token).
pages/	Cuatro vistas principales de la aplicación.

services/	api.ts (instancia Axios) y ecobici.ts (llamadas al backend).
types/	Interfaces TypeScript que modelan las entidades del sistema.

3. Vistas

La aplicación cuenta con cuatro páginas. El acceso a cada una está controlado por el componente ProtectedRoute según el rol del usuario.

Página	Ruta	Acceso	Descripción
LoginPage	/login	Público	Formulario con dos modos (Login / Registro). Al autenticarse redirige al dashboard.
DashboardPage	/dashboard	Todos	Vista en tiempo real del estado de las estaciones. Muestra capacidad, bicicletas disponibles y nivel de ocupación con indicadores de color. Se actualiza automáticamente cada 30 segundos. Incluye búsqueda por nombre e infinite scroll para las más de 600 estaciones.
PrediccionesPage	/predicciones	admin, operador	Muestra la predicción más reciente con sus rutas de redistribución, métricas globales (movimientos mitigados, distancia optimizada, eficiencia) y el estado de cada orden. Los operadores pueden marcar rutas como completadas directamente desde esta vista.

AdminPage	/admin	admin	Dos pestañas: gestión de usuarios (crear, editar rol, activar/desactivar) y registro de auditoría con todas las acciones del sistema, IPs y timestamps. Infinite scroll en el log.
-----------	--------	-------	--

4. Componentes Reutilizables

Navbar

Barra de navegación fija en la parte superior de todas las vistas. Muestra los enlaces de navegación de forma dinámica según el rol del usuario: Estaciones (todos), Predicciones (admin y operador), Admin (solo admin). En el extremo derecho exhibe el nombre del usuario, un badge con su rol y el botón de cierre de sesión.

ProtectedRoute

Componente wrapper que protege las rutas privadas. Mientras el contexto de autenticación está cargando muestra un spinner. Si no hay sesión activa redirige a /login. Si el usuario no tiene el rol requerido lo redirige al dashboard. Solo renderiza los hijos cuando el usuario está autenticado y tiene el rol correcto.

5. Flujo de Autenticación

El token JWT que devuelve el backend al hacer login se almacena en localStorage. AuthContext lo lee al montar la aplicación y llama a GET /auth/me para validarlo y cargar el perfil del usuario. Si la validación falla (respuesta 401), el token se elimina y el usuario queda como no autenticado

El flujo completo de inicio de sesión es el siguiente:

- El usuario ingresa email y contraseña en LoginPage.
- Se realiza POST /auth/login con cuerpo application/x-www-form-urlencoded (requerimiento del esquema OAuth2 de FastAPI).
- El backend devuelve { access_token, token_type }.
- El token se guarda en localStorage y se llama a GET /auth/me.
- El perfil del usuario se almacena en AuthContext y la app redirige al dashboard.

Cada request HTTP posterior incluye automáticamente el header Authorization: Bearer {token} gracias a un interceptor de Axios. Si cualquier respuesta devuelve 401 y hay un token guardado, el interceptor lo elimina y redirige al usuario a /login sin intervención manual.

VI. Estructura de la red

El sistema implementa una arquitectura híbrida compuesta por una fase de procesamiento local y un despliegue escalable en la nube del Tecnológico de Monterrey. La ingesta de datos, el cálculo del buffer de inercia y procesamiento de nuestro modelo de Machine Learning se realizan mediante un servidor físico con sistema operativo Ubuntu Server, interconectado a la red institucional a través de un esquema jerárquico de Router y Switch de acceso.

Por otro lado, los servicios de cara al usuario se encuentran virtualizados en la infraestructura Cloud de la universidad. El tráfico entrante es administrado por una instancia de Balanceo de Carga que distribuye las peticiones hacia alguna de las dos instancias de Frontend, garantizando tolerancia a fallos. La lógica de negocio y los servicios de base de datos operan en instancias Cloud independientes.

VII. Seguridad del Nodo Local (On-Premise)

Para proteger el servidor físico (Ubuntu Server) dentro de la red del campus y asegurar la operación del modelo, se implementó un esquema de endurecimiento (*hardening*) en tres capas críticas:

1. Control de Acceso y SSH Perimetral

- **Autenticación:** Se permite el acceso con contraseña al servidor a través de SSH. Se permite el ingreso mediante Llaves Públicas SSH.
- **Restricción de Superusuario:** Se deshabilitó el acceso directo al usuario root y se cambió el puerto por defecto (22) a un puerto alto discreto para mitigar el escaneo automatizado de vulnerabilidades.

2. Cortafuegos y Tráfico Riguroso

El firewall local opera bajo una política de Denegación por Defecto, limitando las conexiones estrictamente a lo necesario:

- **Entrada (Inbound):** Bloqueada en su totalidad, excepto el puerto SSH personalizado.
- **Salida (Outbound):** Solo se permiten conexiones HTTPS (Puerto 443) hacia las APIs de Ecobici y el Backend Cloud del Tec, además de consultas DNS (Puerto 53).

3. Privilegios Mínimos y Seguridad del Buffer

- **Aislamiento del Proceso:** El script de Machine Learning se ejecuta con un usuario de sistema dedicado.
- **Protección del Archivo de Inercia:** El archivo `buffer_tiempo_real.csv` está restringido con permisos estrictos, garantizando que solo el script pueda leerlo o modificarlo.

- **Defensa Activa:** Se integró **Fail2Ban** para banear automáticamente por IP (vía iptables) a cualquier equipo de la red institucional que registre más de 3 intentos fallidos de conexión.

VIII. Configuración de la Red

https://docs.google.com/document/d/1J0KLSYNz_49ccVw8cVxQnosUy7-o-m3r/edit?usp=sharing&ouid=105904739183663909267&rtpof=true&sd=true